

NomadReady PH — Hackathon Build Specification

1. Build Objective

Build a working hackathon MVP of **NomadReady PH**, an AI-powered LGU readiness dashboard for digital nomad tourism.

The MVP focuses on one pilot destination:

Carles, Iloilo

The system will help an LGU evaluate whether a destination is ready for digital nomads, identify readiness gaps, and generate an AI-assisted action plan.

The hackathon build must clearly show:

1. A dashboard for Carles, Iloilo
 2. A Digital Nomad Readiness Score
 3. Category score breakdown
 4. Local assets such as work spots, accommodations, services, transport, and attractions
 5. A simple map with pins
 6. Top gaps
 7. AI-generated readiness explanation and action plan
-

2. Final Tech Stack

2.1 Backend

Use:

- Django
- Django REST Framework
- PostgreSQL
- OpenAI API
- Python scoring service

Backend responsibilities:

- Store destination and listing data
- Calculate readiness scores
- Serve API endpoints to the frontend
- Generate AI readiness recommendations using OpenAI API
- Store score snapshots

- Store AI recommendation results
 - Seed Carles demo data
-

2.2 Frontend

Use:

- React
- Vite
- Tailwind CSS
- React Router
- Recharts
- Leaflet
- OpenStreetMap

Frontend responsibilities:

- Display the dashboard UI
 - Display score cards and charts
 - Display local asset tables
 - Display map pins
 - Trigger AI recommendation generation
 - Show AI output
-

2.3 Database

Use:

- PostgreSQL

The database will store:

- Destination profile
- Local listings
- Score snapshots
- AI recommendations

For the hackathon, the database will be seeded with demo data for Carles, Iloilo.

2.4 AI

Use:

- OpenAI API

AI feature name:

AI Readiness Advisor

OpenAI will be used to explain the readiness score and generate recommendations.

OpenAI will not calculate the numeric score.

The numeric score must be calculated by the backend rule-based scoring engine.

2.5 Map

Use:

- Leaflet
- OpenStreetMap

The map will display pins for:

- Work spots
 - Accommodations
 - Services
 - Transport
 - Attractions
-

2.6 DevOps

Use:

- Docker
- Docker Compose
- Git
- CI/CD pipeline
- Server deployment from main branch

Services:

- backend

- frontend
 - db
-

3. Team Roles

3.1 Team Lead

Responsibilities:

- Own product scope
 - Maintain this specification
 - Break work into Git issues
 - Review feature branches
 - Approve merges to main
 - Coordinate frontend/backend integration
 - Prepare demo script
 - Coordinate deployment
 - Prevent scope creep
-

3.2 Backend Developer

Responsibilities:

- Set up Django project
 - Set up Django REST Framework
 - Create database models
 - Create migrations
 - Create seed data
 - Build scoring engine
 - Create API endpoints
 - Integrate OpenAI API
 - Add backend Docker setup
 - Add backend tests
 - Support CI/CD deployment
-

3.3 Frontend Developer

Responsibilities:

- Set up React + Vite project
- Set up Tailwind CSS
- Build dashboard layout

- Build score cards and category cards
 - Build local asset table
 - Build map view
 - Build AI Readiness Advisor screen
 - Connect frontend to backend APIs
 - Add responsive layout
 - Add loading and error states
-

4. Git Workflow

Use:

1 feature = 1 branch

Branch naming format:

```
feature/name-of-feature
```

Examples:

```
feature/project-setup  
feature/backend-models-seed  
feature/scoring-engine  
feature/dashboard-overview  
feature/assets-table  
feature/map-view  
feature/ai-readiness-advisor  
feature/cicd-deployment  
feature/demo-polish
```

Workflow:

1. Create feature branch from `main`
2. Build the feature
3. Test locally using Docker
4. Open pull request
5. Team lead reviews
6. Merge to `main` after approval
7. CI/CD deploys latest main branch to server

Rules:

- `main` must always be stable

- unfinished work stays in feature branches
 - no direct push to `main`
 - one feature branch should solve one clear task
 - merge only after review
-

5. Application Modules

The hackathon MVP has 4 main modules:

1. Dashboard Overview
2. Destination Data / Local Assets
3. Map View
4. AI Readiness Advisor

Supporting backend modules:

1. Destination management
 2. Listing management
 3. Scoring engine
 4. OpenAI advisor service
 5. Seed data command
-

6. Main Demo Flow

The hackathon demo must follow this flow:

1. User opens the NomadReady PH dashboard
 2. Dashboard loads Carles, Iloilo data
 3. System shows the overall Digital Nomad Readiness Score
 4. System shows the 5 category scores
 5. User sees top readiness gaps
 6. User opens local assets table
 7. User views work spots, accommodations, services, transport, and attractions
 8. User opens map view
 9. User sees local assets plotted on the map
 10. User opens AI Readiness Advisor
 11. User clicks **Generate AI Recommendations**
 12. OpenAI explains the score and gives an action plan
 13. User sees top recommended actions for the LGU
-

7. Readiness Categories

The hackathon MVP uses 5 categories.

7.1 Internet & Work Readiness

Measures whether digital nomads can work online.

Includes:

- Verified Wi-Fi speed
 - Mobile data coverage
 - Work-friendly cafés
 - Power outlets
 - Zoom-friendly spaces
-

7.2 Long-Stay Accommodation

Measures whether digital nomads can stay for weeks or months.

Includes:

- Monthly stay options
 - Published monthly rates
 - Wi-Fi availability
 - Desk or work area
 - Kitchen or laundry access
-

7.3 Safety & Essential Services

Measures whether the destination has basic support services.

Includes:

- Clinic or hospital
 - Police or emergency contact
 - Pharmacy
 - ATM
 - Laundry
 - Grocery
-

7.4 Transport & Access

Measures whether the destination is easy to reach and move around.

Includes:

- Access from Iloilo City or nearby airport
- Port or terminal availability
- Local transport
- Published schedules or travel information

7.5 Tourism & Lifestyle Appeal

Measures whether the destination is attractive outside working hours.

Includes:

- Island hopping
- Beaches
- Food experiences
- Weekend activities
- Tour operators
- Community or local events

8. Readiness Scoring Formula

Each category is scored from 0 to 100.

The overall score uses weighted category scoring.

Category	Weight
Internet & Work Readiness	30%
Long-Stay Accommodation	20%
Safety & Essential Services	20%
Transport & Access	15%
Tourism & Lifestyle Appeal	15%
Total	100%

Formula:

$$\begin{aligned} \text{Digital Nomad Readiness Score} = & \\ & (\text{Internet \& Work Readiness} \times 0.30) + \\ & (\text{Long-Stay Accommodation} \times 0.20) + \end{aligned}$$

$$\begin{aligned} & (\text{Safety \& Essential Services} \times 0.20) + \\ & (\text{Transport \& Access} \times 0.15) + \\ & (\text{Tourism \& Lifestyle Appeal} \times 0.15) \end{aligned}$$

Score labels:

Score Range	Label
0–39	Not Yet NomadReady
40–59	Emerging Destination
60–74	Developing NomadReady Destination
75–89	NomadReady Destination
90–100	Highly NomadReady Destination

9. Rule-Based Scoring Requirements

Only `lgu_verified` listings count toward scoring.

Draft listings do not increase the score.

Missing data lowers the score.

Category scores are capped at 100.

Overall score is rounded to the nearest whole number.

9.1 Internet & Work Readiness

Maximum: 100 points

Base score:

Condition	Points
5+ verified work-friendly places with Wi-Fi	30
3–4 verified work-friendly places with Wi-Fi	20
1–2 verified work-friendly places with Wi-Fi	10
No verified work-friendly places	0

Additional points:

Condition	Points
At least 3 places publish Wi-Fi speed	25
At least 3 places are Zoom-friendly	20
At least 3 places have power outlets	15
Mobile data coverage is available	10

9.2 Long-Stay Accommodation

Maximum: 100 points

Base score:

Condition	Points
5+ long-stay options	35
3-4 long-stay options	25
1-2 long-stay options	10
No long-stay options	0

Additional points:

Condition	Points
Monthly rates are published	25
Listings have Wi-Fi	20
Listings have desk, kitchen, or laundry access	20

9.3 Safety & Essential Services

Maximum: 100 points

Condition	Points
Clinic or hospital is listed	25

Condition	Points
Police or emergency contact is listed	20
Pharmacy is listed	15
ATM is listed	15
Laundry or grocery is listed	15
Safety notes are available	10

9.4 Transport & Access

Maximum: 100 points

Condition	Points
Main access route from Iloilo City is listed	30
Port or terminal is listed	25
Local transport option is listed	20
Schedules or fare information is available	15
Travel time is shown	10

9.5 Tourism & Lifestyle Appeal

Maximum: 100 points

Base score:

Condition	Points
5+ attractions or activities are listed	30
3-4 attractions or activities are listed	20
1-2 attractions or activities are listed	10
No attractions listed	0

Additional points:

Condition	Points
Island hopping or beach activity is available	25
Tour operator is listed	20
Food, culture, nature, or wellness activity is listed	15
Event or community activity is listed	10

10. Backend Specification

10.1 Backend Structure

Backend folder:

```
backend/
```

Django apps:

```
apps/  
  destinations/  
  listings/  
  scoring/  
  ai_advisor/
```

destinations

Handles destination profile.

listings

Handles work spots, accommodations, services, transport, and attractions.

scoring

Handles rule-based score calculation.

ai_advisor

Handles OpenAI recommendation generation.

10.2 Database Models

Destination

Fields:

- id
 - name
 - province
 - municipality
 - description
 - created_at
 - updated_at
-

Listing

Fields:

- id
- destination
- name
- category
- type
- address
- latitude
- longitude
- details
- verification_status
- created_at
- updated_at

Listing category options:

- work_spot
- accommodation
- service
- transport
- attraction

Verification status options:

- draft
- lgu_verified
- needs_update

Use a JSON field for flexible `details`.

Example `details` JSON for work spot:

```
{
  "wifi_speed_mbps": 50,
  "has_wifi": true,
  "has_power_outlets": true,
  "zoom_friendly": true,
  "mobile_data_available": true
}
```

Example `details` JSON for accommodation:

```
{
  "monthly_rate": 18000,
  "has_wifi": true,
  "has_desk": true,
  "has_kitchen": true,
  "has_laundry": false,
  "long_stay_friendly": true
}
```

Example `details` JSON for service:

```
{
  "service_type": "clinic",
  "contact_number": "sample only",
  "safety_notes_available": true
}
```

Example `details` JSON for transport:

```
{
  "transport_type": "port",
  "has_schedule_info": true,
  "has_fare_info": true,
  "travel_time_from_iloilo": "4-5 hours"
}
```

Example `details` JSON for attraction:

```
{
  "activity_type": "island_hopping",
}
```

```
"weekend_friendly": true,  
"tour_operator_available": true  
}
```

ScoreSnapshot

Fields:

- id
- destination
- overall_score
- score_label
- internet_work_score
- accommodation_score
- safety_services_score
- transport_score
- tourism_lifestyle_score
- strongest_category
- weakest_category
- top_gaps
- created_at

Use JSON field for `top_gaps`.

AIRecommendation

Fields:

- id
- destination
- score_snapshot
- title
- affected_category
- recommendation
- priority
- reason
- generated_at

Priority options:

- high
 - medium
 - low
-

10.3 API Endpoints

Base URL:

```
/api/
```

Destination Endpoint

GET `/api/destinations/carles/`

Returns the Carles destination profile.

Response:

```
{
  "id": 1,
  "name": "Carles",
  "province": "Iloilo",
  "municipality": "Carles",
  "description": "Gateway to island destinations including Gigantes Islands."
}
```

Listings Endpoint

GET `/api/listings/`

Returns all listings.

Query parameters:

- category
- verification_status

Example:

```
GET /api/listings/?category=work_spot
```

Response:

```
[
  {
    "id": 1,
    "name": "Carles Port Café",
    "category": "work_spot",
    "type": "cafe",
    "address": "Carles, Iloilo",
    "latitude": 11.571,
    "longitude": 123.134,
    "verification_status": "lgu_verified",
    "details": {
      "wifi_speed_mbps": 50,
      "has_wifi": true,
      "has_power_outlets": true,
      "zoom_friendly": true
    }
  }
]
```

POST /api/listings/

Creates a listing.

For the hackathon, this is optional. Seeded demo data is enough.

PATCH /api/listings/{id}/

Updates a listing.

For the hackathon, this is optional. Seeded demo data is enough.

Scoring Endpoints

GET /api/scores/current/

Returns latest readiness score.

Response:

```
{
  "overall_score": 68,
  "score_label": "Developing NomadReady Destination",
  "categories": {
```

```
    "internet_work": 55,
    "accommodation": 60,
    "safety_services": 75,
    "transport": 70,
    "tourism_lifestyle": 90
  },
  "strongest_category": "Tourism & Lifestyle Appeal",
  "weakest_category": "Internet & Work Readiness",
  "top_gaps": [
    "Limited verified work-friendly places",
    "Not enough published Wi-Fi speed data",
    "Few monthly stay packages"
  ]
}
```

POST /api/scores/recalculate/

Recalculates readiness score from current listings and creates a new ScoreSnapshot.

Response:

```
{
  "message": "Score recalculated",
  "overall_score": 68,
  "score_label": "Developing NomadReady Destination"
}
```

AI Advisor Endpoint

POST /api/ai-advisor/generate/

Generates AI readiness explanation and recommendations using OpenAI API.

Input:

```
{
  "destination_id": 1
}
```

Response:

```
{
  "summary": "Carles has strong tourism appeal but still needs more verified work-friendly infrastructure.",
  "strengths": [
    "Strong island-hopping and weekend tourism appeal",
    "Existing transport access through port routes"
  ],
  "weaknesses": [
    "Limited verified work-friendly spaces",
    "Insufficient published Wi-Fi speed data"
  ],
  "recommendations": [
    {
      "title": "Certify 3 to 5 work-friendly cafés or hotels",
      "affected_category": "Internet & Work Readiness",
      "priority": "high",
      "reason": "Digital nomads need reliable places to work during weekdays.",
      "suggested_next_step": "Create an LGU checklist for cafés and hotels that includes Wi-Fi speed, power outlets, noise level, and opening hours."
    }
  ]
}
```

11. Scoring Service Implementation

Create:

```
backend/apps/scoring/services.py
```

Required functions:

```
calculate_internet_work_score(listings) -> int
calculate_accommodation_score(listings) -> int
calculate_safety_services_score(listings) -> int
calculate_transport_score(listings) -> int
calculate_tourism_lifestyle_score(listings) -> int
calculate_overall_score(category_scores) -> dict
get_score_label(overall_score) -> str
get_top_gaps(category_scores, listings) -> list[str]
calculate_destination_score(destination_id) -> ScoreSnapshot
```

Rules:

- Only `lgu_verified` listings count toward scoring.
 - Draft listings do not increase the score.
 - Missing data lowers the score.
 - Category scores are capped at 100.
 - Overall score is rounded to the nearest whole number.
 - ScoreSnapshot is created every time scoring is recalculated.
-

12. OpenAI Advisor Service Implementation

Create:

```
backend/apps/ai_advisor/services.py
```

Required function:

```
generate_readiness_advice(destination_id, score_snapshot_id) -> dict
```

The OpenAI prompt must include:

- Destination name
- Overall score
- Score label
- Category scores
- Strongest category
- Weakest category
- Top gaps
- Verified listing summary
- Missing data

The OpenAI output must include:

- summary
- strengths
- weaknesses
- recommendations

The OpenAI output should be parsed and returned as structured JSON.

OpenAI guardrails:

- Do not invent places.
- Do not invent statistics.

- Do not change the score.
 - Mention missing data clearly.
 - Recommendations must be practical for an LGU.
 - Tone must be professional, constructive, and government-friendly.
 - Output must focus on actions the tourism office can take.
-

13. Frontend Specification

Frontend folder:

```
frontend/
```

Use:

- React
 - Vite
 - Tailwind CSS
 - React Router
 - Recharts
 - Leaflet
 - OpenStreetMap
-

13.1 Frontend Routes

Routes:

```
/
/dashboard
/assets
/map
/ai-advisor
```

Route behavior:

- `/` redirects to `/dashboard`
 - `/dashboard` shows the dashboard overview
 - `/assets` shows destination data and local assets
 - `/map` shows map pins
 - `/ai-advisor` shows OpenAI-generated readiness explanation and action plan
-

13.2 Layout

Use one main dashboard layout.

Layout components:

- Sidebar navigation
- Top bar
- Main content area
- Cards
- Tables
- Map panel
- AI recommendation panel

Sidebar items:

- Dashboard
- Local Assets
- Map View
- AI Advisor

14. Screen Specifications

14.1 Screen 1: Dashboard Overview

Route:

/dashboard

Purpose:

Show the main readiness status of Carles.

Required components:

1. Header
2. Title: Carles Digital Nomad Readiness
3. Subtitle: LGU Dashboard for NomadReady PH
4. Overall Score Card

5. Score number
6. Score label
7. Short status text
8. Category Score Cards
9. Internet & Work Readiness
10. Long-Stay Accommodation
11. Safety & Essential Services
12. Transport & Access
13. Tourism & Lifestyle Appeal
14. Top Gaps Card
15. Show top 3 gaps
16. AI Recommendations Preview
17. Show top 3 recommendations if already generated
18. Button to open AI Advisor
19. Key Metrics Cards
20. Verified work-friendly places
21. Long-stay options
22. Essential services
23. Attractions and activities

Acceptance criteria:

- Dashboard loads Carles data.
 - Dashboard displays overall score.
 - Dashboard displays all 5 category scores.
 - Dashboard displays score label.
 - Dashboard displays top gaps.
 - Dashboard has navigation to AI Advisor.
 - Dashboard is usable on a laptop screen.
-

14.2 Screen 2: Destination Data / Local Assets

Route:

/assets

Purpose:

Show the data used for scoring.

Required components:

1. Page header
2. Title: Destination Data / Local Assets
3. Subtitle: Verified local data used for readiness scoring
4. Category filters
5. All
6. Work Spots
7. Accommodations
8. Services
9. Transport
10. Attractions
11. Listings table
12. Name
13. Category
14. Type
15. Location
16. Key details
17. Verification status
18. Optional add/edit button
19. For hackathon, button can open a simple modal or be disabled with "Seeded demo data"

Acceptance criteria:

- User can view local asset listings.
- User can filter by category.
- Verification status is visible.
- Listings match the scoring categories.
- Seeded demo data is enough.

14.3 Screen 3: Map View

Route:

/map

Purpose:

Show local assets on a simple map.

Required components:

1. Page header
2. Title: Carles Local Asset Map
3. Leaflet map
4. Centered on Carles, Iloilo
5. Pins for listings
6. Pin categories
7. Work spots
8. Accommodations
9. Services
10. Transport
11. Attractions
12. Map legend
13. Different icon or label per category

14. Popup on pin click

15. Name

16. Type

17. Verification status

18. Key details

Acceptance criteria:

- Map loads without crashing.
- Map shows seeded pins.
- User can click a pin and view listing details.
- Map visually supports the dashboard story.

14.4 Screen 4: AI Readiness Advisor

Route:

/ai-advisor

Purpose:

Generate and show OpenAI explanation and LGU action plan.

Required components:

1. Page header
2. Title: AI Readiness Advisor
3. Subtitle: AI-generated explanation and action plan based on verified destination data
4. Score Summary Panel
5. Overall score
6. Score label
7. Strongest category
8. Weakest category
9. Generate Button

10. Label: Generate AI Recommendations

11. AI Output Panel

12. Readiness summary

13. Strengths

14. Weaknesses

15. Top 3 recommended actions

16. Recommendation Cards

17. Title

18. Affected category

19. Priority

20. Reason

21. Suggested next step

Acceptance criteria:

- User can click Generate AI Recommendations.
- Frontend calls backend AI endpoint.
- Loading state is shown.
- AI result is displayed.
- AI result does not alter the numeric score.
- AI result references real category scores and gaps.

15. Demo Data Requirements

Seed enough data to produce an interesting score.

The score should not be too high. It should show opportunity.

Target demo score:

Overall Score: 68 / 100

Label: Developing NomadReady Destination

Target category scores:

Category	Target Score
Internet & Work Readiness	55
Long-Stay Accommodation	60
Safety & Essential Services	75
Transport & Access	70
Tourism & Lifestyle Appeal	90

Demo story:

- Carles is strong in tourism.
- Carles has decent access and services.
- Carles needs better work-ready infrastructure and long-stay packaging.

15.1 Example Seed Listings

Work spots:

- Carles Port Café
- Isla Work Lounge
- Gigantes Gateway Inn Lobby

Accommodations:

- Carles Bay Homestay
- Gigantes Gateway Pension
- Island Stay Carles

Services:

- Carles Municipal Health Office
- Carles Police Station
- Local Pharmacy
- Laundry Service
- ATM near town center

Transport:

- Iloilo City to Carles van route
- Bancal Port
- Tricycle local transport
- Boat route to Gigantes Islands

Attractions:

- Gigantes Islands
- Cabugao Gamay Island
- Bantigue Sandbar
- Antonia Beach
- Seafood food experience

These are demo placeholders. Mark them as sample data where needed.

16. Docker Specification

Use Docker Compose.

Services:

```
backend
frontend
db
```

Optional later:

```
nginx
```

16.1 Backend Container

Runs:

```
Django app
```

Port:

```
8000
```

Backend command:

```
python manage.py runserver 0.0.0.0:8000
```

16.2 Frontend Container

Runs:

React Vite app

Development port:

5173

Frontend command:

npm run dev -- --host 0.0.0.0

16.3 Database Container

Runs:

PostgreSQL

Port:

5432

Environment variables:

```
POSTGRES_DB=nomadready
POSTGRES_USER=nomadready
POSTGRES_PASSWORD=nomadready_dev
DATABASE_URL=postgres://nomadready:nomadready_dev@db:5432/nomadready
OPENAI_API_KEY=
DJANGO_SECRET_KEY=
DJANGO_DEBUG=True
```

17. CI/CD Specification

Use a simple CI/CD flow.

Trigger:

```
push to main
```

Pipeline steps:

1. Checkout code
2. Build backend container
3. Build frontend container
4. Run backend tests
5. Run frontend build
6. Deploy to server
7. Restart Docker Compose services

Minimum backend checks:

```
python manage.py check  
python manage.py test
```

Minimum frontend checks:

```
npm install  
npm run build
```

Deployment command on server:

```
git pull origin main  
docker compose down  
docker compose up -d --build  
docker compose exec backend python manage.py migrate  
docker compose exec backend python manage.py seed_demo_data
```

CI/CD must keep the main branch deployable.

18. Testing Requirements

18.1 Backend Tests

Required tests:

1. Score label test
 2. Internet & Work Readiness scoring test
 3. Accommodation scoring test
 4. Safety & Services scoring test
 5. Transport scoring test
 6. Tourism scoring test
 7. Overall score calculation test
 8. Current score API test
 9. Listings API test
 10. AI Advisor endpoint mock test
-

18.2 Frontend Manual Tests

Required manual checks:

1. Dashboard loads
 2. Score cards display
 3. Category scores display
 4. Asset table filters work
 5. Map loads
 6. Pins appear
 7. Pin popups display listing details
 8. AI button triggers API call
 9. Loading state appears
 10. AI recommendations display
 11. Layout works on laptop screen
-

19. Feature Branch Plan

Branch 1: Project Setup

Branch:

feature/project-setup

Tasks:

- Create repo structure
- Add backend Django project
- Add frontend React app
- Add Docker Compose
- Add PostgreSQL service
- Confirm local development runs

Done when:

- Backend runs
- Frontend runs
- Database runs
- README has setup instructions

Branch 2: Backend Models and Seed Data

Branch:

feature/backend-models-seed

Tasks:

- Create Destination model
- Create Listing model
- Create ScoreSnapshot model
- Create AIRecommendation model
- Add migrations
- Add seed command for Carles demo data

Done when:

- Database migrates
- Seed data loads
- API or Django shell can view data

Branch 3: Scoring Engine

Branch:

feature/scoring-engine

Tasks:

- Implement scoring service
- Implement score snapshot creation
- Add score API endpoints
- Add tests

Done when:

- Current score API returns overall score
 - Category scores are calculated
 - Tests pass
-

Branch 4: Dashboard Overview UI

Branch:

feature/dashboard-overview

Tasks:

- Create layout
- Create dashboard route
- Add score cards
- Add category cards
- Add top gaps card
- Connect to score API

Done when:

- Dashboard shows real backend score data
-

Branch 5: Assets Table UI

Branch:

feature/assets-table

Tasks:

- Create assets route
- Create listing table
- Add category filters
- Connect to listings API

Done when:

- User can view and filter local assets
-

Branch 6: Map View

Branch:

feature/map-view

Tasks:

- Install Leaflet
- Create map route
- Plot seeded listing pins
- Add popup details
- Add legend

Done when:

- Map displays pins and popups
-

Branch 7: AI Readiness Advisor

Branch:

feature/ai-readiness-advisor

Tasks:

- Add OpenAI backend service
- Add AI endpoint
- Add frontend AI Advisor page
- Add generate button
- Display AI recommendations

Done when:

- User can generate and view AI readiness advice
-

Branch 8: CI/CD and Deployment

Branch:

feature/cicd-deployment

Tasks:

- Add CI/CD config
- Add production Docker Compose
- Configure server deployment
- Add environment variables
- Test push to main deployment

Done when:

- Merge to main deploys working app
-

Branch 9: Demo Polish

Branch:

feature/demo-polish

Tasks:

- Improve UI spacing
- Add final copy
- Add loading states
- Add empty states
- Add sample data labels
- Prepare final presentation flow

Done when:

- App is demo-ready for judges
-

20. Definition of Done

A feature is done when:

- It matches this specification
 - It runs locally in Docker
 - It does not break existing screens
 - Backend tests pass, if backend feature
 - Frontend build passes, if frontend feature
 - Team lead has reviewed it
 - It has been merged to main
 - Main branch remains deployable
-

21. Hackathon MVP Acceptance Criteria

The full MVP is accepted when:

- The app runs using Docker Compose
 - Backend API is working
 - Frontend dashboard is working
 - Database has seeded Carles demo data
 - Dashboard shows overall readiness score
 - Dashboard shows 5 category scores
 - Asset table displays local listings
 - Map shows local assets as pins
 - AI Readiness Advisor generates explanation and recommendations using OpenAI API
 - CI/CD can deploy latest main branch
 - Demo story is clear within 3 to 5 minutes
-

22. Instructions for Claude Code

Claude Code must follow this specification.

Rules for Claude Code:

1. Do not expand beyond the defined hackathon scope.
2. Use Django and Django REST Framework for backend.
3. Use React, Vite, Tailwind CSS, Recharts, and Leaflet for frontend.
4. Use PostgreSQL as the database.
5. Use OpenAI API for AI Readiness Advisor.
6. Do not let OpenAI calculate the numeric score.
7. Implement numeric scoring in deterministic backend Python code.
8. Use seeded Carles demo data.
9. Keep the app working in Docker.

10. Use one feature branch per feature.
11. Keep main branch deployable.
12. Add backend tests for scoring.
13. Build only the 4 MVP screens:
 - Dashboard Overview
 - Destination Data / Local Assets
 - Map View
 - AI Readiness Advisor

Do not build:

- Full digital nomad public app
- Login system unless required for deployment
- Business owner portal
- Booking system
- Review system
- Real-time Wi-Fi testing
- Public destination pages
- Multi-LGU comparison
- Advanced admin panel

23. Final Build Focus

The winning MVP is simple:

A working LGU dashboard that turns local tourism data into a Digital Nomad Readiness Score and an AI-generated LGU action plan.

Final demo message:

NomadReady PH helps LGUs prepare for the future of tourism by showing whether a destination is ready for digital nomads, what gaps need to be fixed, and what actions should be taken first.

Pilot destination:

Carles, Iloilo — from island tourism destination to digital nomad-ready destination.